

Question

The current selected programming language is **C++14**. We emphasize the submission of a fully working code over partially correct but efficient code. Once **submitted**, you cannot review this problem again. You can use `cout` to debug your code. The `cout` may not work in case of syntax/runtime error. The version of **GCC** being used is **5.5.0**.

An IT park consists of similar buildings. During the initial construction of the IT park, the telecom company "Nazania Communication" used some of the buildings for its servers. Now that the IT park has been fully developed, all the buildings need internet connections. The telecom company wants to connect each building to the nearest building where a server has already been set. To plan for this, the company creates a rectangular aerial view of the buildings in the IT park in the form of a $M \times N$ grid. Each cell of the grid represents a building. For all the buildings that do not have servers, the company wants to find the minimum number of buildings (horizontally and/or vertically) that separate the buildings that need internet connections from buildings that have servers.

Write an algorithm to find the minimum number of buildings (horizontally and/or vertically) that separate the buildings that need internet connections from buildings that have servers.

Input

The first line of the input consists of two space-separated integers, *rows* and *columns*, representing the number of rows(M) and the number of columns(N) in the grid.

The next M lines consist of N space-separated integers representing the aerial view, where 1 represents a server building and 0 represents a non-server building.

Output

Print N space-separated integers in M lines representing the aerial view where each element consists of the minimum number of buildings (horizontally and/or vertically) that separate the buildings that need internet connections from buildings that have servers.

Constraints

$$1 \leq M < 100$$

$$1 \leq N < 100$$

```

1 // Header Files
2 #include<iostream>
3 #include<string>
4 #include<vector>
5 using namespace std;
6
7
8 /*
9  * aerialView, representing the
10  */
11 vector<vector<int> > closestDis
12 {
13     vector<vector<int> > answer;
14     // Write your code here
15
16
17     return answer;
18 }
19
20 int main()
21 {
22     //input for aerialView
23     int aerialView_row;
24     int aerialView_col;
25     cin >> aerialView_row;
26     cin >> aerialView_col;
27
28     vector<vector<int> >
29     for ( int idx = 0; i
30     {
31         vector<int> temp
32         for ( int jdx =
33         {
34             int temp;
35             cin >> tem
36             temp_vecto
37         }
38         aerialView.pu
39     }
40
41
42     vector<vector<int> >

```

Sheldon is going to a book fair where all the books are star-rated. As he is interested in just two types of books, Horror and Sci-fi, so he would buy the books from these two categories only. He would want to buy at least one book from each category so as to maximize the total star-rating of his books. Also, the total price of the books should not exceed the amount of money that he can spend. The output is -1 if it is not possible to buy at least one book from both the categories with the money that he has.

Write an algorithm to help Sheldon buy the books from both the categories.

Input

The first line of the input consists of an integer - *amount*, representing the amount of money Sheldon can spend.

The second line consists of two integers - *numHorror* and *numH*, representing the number of Horror books (H) and the number of values given for every horror book (X is always equal to 2), respectively.

The next H lines consist of X space-separated integers - *hrating* and *hprice*, representing the star-rating and the price of each Horror book, respectively.

The next line consists of two space-separated integers - *numSciFi* and *numS*, representing the number of Sci-fi books (S) and the number of values given for every Sci-fi book (P is always equal to 2), respectively.

The last S lines consist of P space-separated integers - *srating* and *sprice*, representing the star-rating and the price of each Sci-fi book, respectively.

Output

```
5 using n
6
7
8 /*
9  * amo
10 horror
11 sciFiB
12 */
13 int ma
14 {
15     i
16
17
18
19
20 }
21
22 int
23 {
24
25
26
```


Output

Print an integer representing the total maximum star-rating of books bought by Sheldon. If he cannot buy at least one book from both the categories then print -1.

Constraints

$$1 \leq \text{numHorror} \leq 1000$$

$$1 \leq \text{numSciFi} \leq 1000$$

$$1 \leq \text{amount} \leq 10^5$$

$$1 \leq \text{hrating}, \text{srating} \leq 10^6$$

$$1 \leq \text{hprice}, \text{sprice} \leq 10^5$$

$$\text{numH} = 2$$

$$\text{numS} = 2$$

Example

Input:

50

3 2

5 10

3 30

6 20

3 2

6 30

4 30

2 10

Output:

13

Explanation:

Select 1st and 3rd book from Horror books.

Select 3rd book from Sci-fi books.

(Total Price = $10 + 20 + 10 = 40$) < (amount = 50)

```
1 // Header Files
2 #include<iostream>
3 #include<string>
4 #include<vector>
5 using namespace std;
6
7
8 /*
9  * amount representing the am
10 horrorBooks is a grid where e
11 sciFiBooks is a grid where ea
12 */
13 int maxRatingBooks (int amou
14 {
15     int answer;
16     // Write your code here
17
18
19     return answer;
20 }
21
22 int main()
23 {
24     //input for amount
25     int amount;
26     cin >> amount;
27
28     //input for horrorBoo
29     int horrorBooks_row;
30     int horrorBooks_col;
```

3 2

5 10

3 30

6 20

3 2

6 30

4 30

2 10

Output:

13

Explanation:

Select 1st and 3rd book from Horror books.

Select 3rd book from Sci-fi books.

(Total Price = $10 + 20 + 10 = 40$) < (amount = 50)

Maximum star-rating = $5 + 6 + 2 = 13$

So, the output is 13.



Show tips

```
4  #include<vector>
5  using namespace std;
6
7
8  /*
9   * amount representing the amount of money Sheldon can spend.
10  horrorBooks is a grid where each row represents the star-rating and the price of a Horror book.
11  sciFiBooks is a grid where each row represents the star-rating and the price of a Sci-Fi book.
12  */
13  int maxRatingBooks (int amount, vector<vector<int> > horrorBooks, vector<vector<int> > sciFiBooks)
14  {
15      int answer;
16      // Write your code here
17
18
19      return answer;
20  }
21
22  int main()
23  {
24      //input for amount
25      int amount;
26      cin >> amount;
27
```

An internet service provider company Si-Fi has won a city tender to lay the wire cables for data transmission between N given routes. There is a chance that the two routes may intersect at some point. Because intersecting wires can cause signal distortion, the company decides to place a distortion avoidance device at all such locations. Find the number of devices the company needs to place.

Write an algorithm to find the number of distortion avoidance devices the company needs to place.

Input

The first line of the input consists of two space-separated integers - *num* and *coordinateCount*, representing the number of routes (N) and the number of coordinates of the two locations connected by a route (M is always equal to 4). The next N lines consist of M space-separated integers representing the respective x and y coordinates of the two locations that are connected to each other.

Output

Print an integer representing the number of distortion avoidance devices the company needs to place.

Constraints

$$0 \leq \text{num} \leq 10^4$$

$0 \leq x, y \leq 10^9$; x and y represents x and y coordinates of a location

```
6
7
8  /*
9  * locCord
10 */
11 int numDev
12 {
13     int a
14     // writ
15
16
17     return
18 }
19
20 int main()
21 {
22     //input
23     int loc
24     int loc
25     cin >>
26     cin >>
27
28     vector<
29     for ( i
30     {
31         vect
```


two locations connected by a route (M is always equal to 4).
The next N lines consist of M space-separated integers representing the respective x and y coordinates of the two locations that are connected to each other.

Output

Print an integer representing the number of distortion avoidance devices the company needs to place.

Constraints

$$0 \leq \text{num} \leq 10^4$$

$0 \leq x, y \leq 10^9$; x and y represents x and y coordinates of a location

$$\text{coordinateCount} = 4$$

Example

Input:

3 4
1 1 4 5
0 3 3 2
2 4 5 3

Output:

2

Explanation:

The route of the coordinates (1,1), (4,5) intersects with the routes of the coordinates (0,3), (3,2) and (2,4), (5,3).

Two distortion avoidance devices must be placed here.

So, the output is 2.

```
4 #include<vector>
5 using namespace std;
6
7
8 /*
9  * locCord is a vector<vector<int>>
10  */
11 int numDevices (vector<vector<int>> &locCord)
12 {
13     int answer;
14     // Write your code here
15
16
17     return answer;
18 }
19
20 int main()
21 {
22     //input for locCord
23     int locCord_row;
24     int locCord_col;
25     cin >> locCord_row;
26     cin >> locCord_col;
27
28     vector<vector<int>> locCord;
29     for ( int idx = 0; idx < locCord_row; idx++)
30     {
31         vector<int> row;
```

```

5 using namespace std;
6
7
8 /*
9  * locCord is a matrix where each row representing x and y coordinates of the two locations that are connected
10  */
11 int numDevices (vector<vector<int> > locCord)
12 {
13     int answer;
14     // Write your code here
15
16
17     return answer;
18 }
19
20 int main()
21 {
22     //input for locCord
23     int locCord_row;
24     int locCord_col;
25     cin >> locCord_row;
26     cin >> locCord_col;
27
28     vector<vector<int> > locCord;
29     for ( int idx = 0; idx < locCord_row; idx++ )
30     {
31         vector<int> temp vector;

```

Test Cases & Output

RUN SOLUTION

SUB

ENG



Question

Tim, a seventh grade student, is introduced to the concept of lines in basic geometry class. He applies this concept in real life. If he considers his house as one point and his friend Bill's house as another point, he can draw a line between these two points. Similarly, if the houses of all his friends are considered as different points, he can draw multiple lines with his own house as the common point in each line. By taking his school as the reference, Tim marks the coordinates of his N friends' houses.

Write an algorithm to help Tim that takes the coordinates of his house (x_0 , y_0) and his friends' houses (x_i , y_i) and outputs the number of unique lines that can be drawn with Tim's house as the common point in each line.

Input

The first line of the input consists of two space-separated integers - n and $numCoordinate$, representing the number of Tim's friends (N), number of coordinates in each of the house position (M is always equal to 2), respectively. The next N lines consist of two space-separated integers - $frndXi$ and $frndYi$ representing the X-coordinates and the Y-coordinates of the houses of Tim's friends. The next line consists of an integer - $timX0$, representing the X-coordinate of Tim's house. The last line consists of an integer -

```

1 // Header Files
2 #include<iostream>
3 #include<string>
4 #include<vector>
5 using namespace std;
6
7
8 /*
9  * frndCoord is list of XY-coordinates of friends houses.
10 timX is integer representing the X-coordinate of Tim's house.
11 timY is integer representing the Y-coordinate of Tim's house.
12 */
13 int minLines (vector<vector<int> > frndCoord, int timX, int timY)
14 {
15     int answer;
16     // Write your code here
17
18
19     return answer;
20 }
21
22 int main()
23 {
24     //input for frndCoord
25     int frndCoord_row;
26     int frndCoord_col;
27     cin >> frndCoord_row;
28     cin >> frndCoord_col;
29
30     vector<vector<int> > frndCoord;
31     for ( int idx = 0; idx < frndCoord_row; idx++ )
32     {
33         vector<int> temp_vector;
34         for ( int jdx = 0; jdx < frndCoord_col; jdx++ )
35         {
36             int temp;
37             cin >> temp;
38             temp_vector.push_back(temp);
39         }
40         frndCoord.push_back(temp_vector);
41     }
42

```

Question

The current selected programming language is **C++14**. We emphasize the submission of a fully working code over partially correct but efficient code. Once **submitted**, you cannot review this problem again. You can use `cout` to debug your code. The `cout` may not work in case of syntax/runtime error. The version of **GCC** being used is **5.5.0**.

Given a binary string `S` consisting of only 0s and 1s, write an algorithm to find the number of different ways to get the longest consecutive sub-segment of 1s only. You are allowed to change any `K` number of 0s to 1s. If two ways lead to the same string, they are considered to be similar, not different.

Input

The first line of the input consists of the string (`S`).
The second line consists of an integer - `changeK`, representing the number of changes you can make (`K`).

```
1 // Header Files
2 #include<iostream>
3 #include<string>
4 #include<vector>
5 using namespace std;
6
7
8 /*
9  * strS is the given string.
10  * changeK is the number of changes you can make.
11  */
12 int longSubseg (string strS, int changeK)
13 {
14     int answer;
15     // Write your code here
16
17
18     return answer;
19 }
20
21 int main()
22 {
23
24     //input for strS
25     string strS;
26     getline(cin, strS);
27     //input for changeK
28     int changeK;
29     cin >> changeK;
30
31
32     int result = longSubseg(strS, changeK);
```

Test

29°C
Mostly cloudy

Search

ASUS Vivobook

Question

CLINT'S SOLUTIONS

Input

The first line of the input consists of the string (S).

The second line consists of an integer - *changeK*, representing the number of changes you can make (K).

Output

Print an integer representing the number of different ways possible to get the longest consecutive sub-segment of 1s.

Constraints

$1 \leq \text{size of string} \leq 2 \cdot 10^5$

$0 \leq \text{changeK} \leq \text{size}$

Example

Input:

1010101

1

Output:

3

Explanation:

29°C
Mostly cloudy

```
1 // Header Files
2 #include<iostream>
3 #include<string>
4 #include<vector>
5 using namespace std;
6
7
8 /*
9  * strS is the given string.
10  * changeK is the number of changes you can make.
11  */
12 int longSubseg (string strS, int changeK)
13 {
14     int answer;
15     // Write your code here
16
17
18     return answer;
19 }
20
21 int main()
22 {
23
24     //input for strS
25     string strS;
26     getline(cin, strS);
27     //input for changeK
28     int changeK;
29     cin >> changeK;
30
31
32     int result = longSubseg(strS, changeK);
```



Search



ASUS Vivobook

Question

Output

Print an integer representing the number of different ways possible to get the longest consecutive sub-segment of 1s.

Constraints

$1 \leq \text{size of string} \leq 2 \cdot 10^5$
 $0 \leq \text{changeK} \leq \text{size}$

Example

Input:

1010101

1

Output:

3

Explanation:

Since only one change is allowed the different combinations possible are 1110101, 1011101 and 1010111. In all the possible combinations the maximum reading with consecutive 1s is in 1011101 i.e 111 so 3 is the output.

```
1 // Header Files
2 #include<iostream>
3 #include<string>
4 #include<vector>
5 using namespace std;
6
7
8 /*
9  * strS is the given string.
10  * changeK is the number of changes you can make.
11  */
12 int longSubseg (string strS, int changeK)
13 {
14     int answer;
15     // Write your code here
16
17     return answer;
18 }
19
20
21 int main()
22 {
23
24     //input for strS
25     string strS;
26     getline(cin, strS);
27     //input for changeK
28     int changeK;
29     cin >> changeK;
30
31
32     int result = longSubseg(strS, changeK);
33 }
```

```

1 // Header Files
2 #include<iostream>
3 #include<string>
4 #include<vector>
5 using namespace std;
6
7
8 /*
9  * strS is the given string.
10 changeK is the number of changes you can make.
11 */
12 int longSubseg (string strS, int changeK)
13 {
14     int answer;
15     // Write your code here
16
17     return answer;
18 }
19
20
21 int main()
22 {
23
24     //input for strS
25     string strS;
26     getline(cin,strS);
27     //input for changeK
28     int changeK;
29     cin >> changeK;
30
31
32     int result = longSubseg(strS, changeK);

```

Test Cases & Output

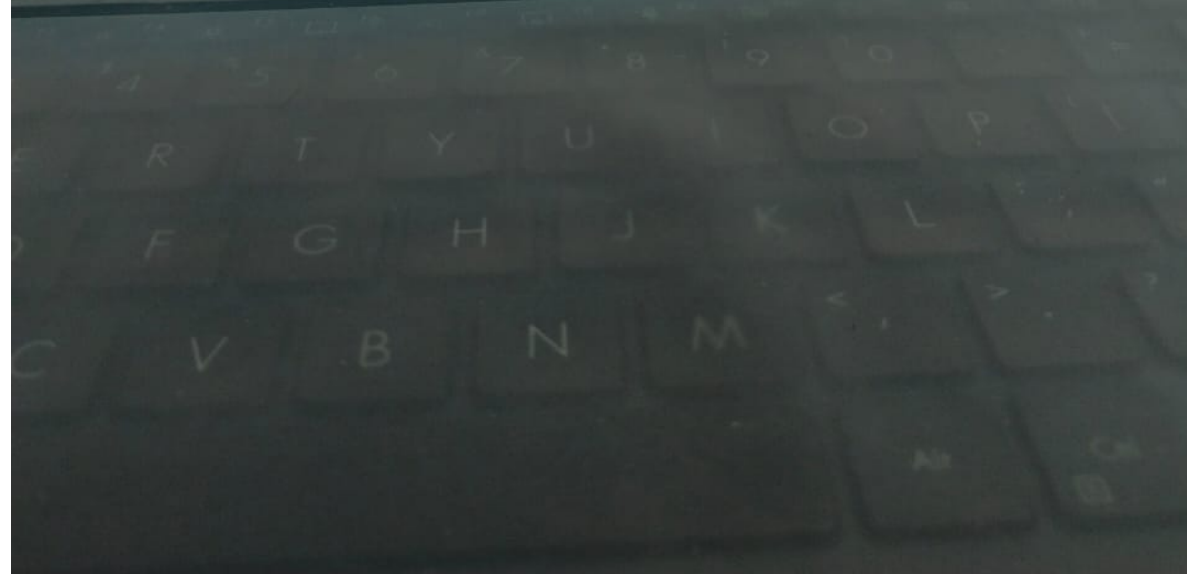
RUN SOLUTION



Search



ASUS Vivobook



Code saved

```
7
8 * /*
9  * strS is the given string.
10 changeK is the number of changes you can make.
11 */
12 int longSubseg (string strS, int changeK)
13 {
14     int answer;
15     // Write your code here
16
17
18     return answer;
19 }
20
21 int main()
22 {
23
24     //input for strS
25     string strS;
26     getline(cin, strS);
27     //input for changeK
28     int changeK;
29     cin >> changeK;
30
31
32     int result = longSubseg(strS, changeK);
33     cout << result;
34
35
36     return 0;
37 }
38
```

Test Cases & Output

RUN SOLUTION



Search



ASUS Vivobook

Question

The current selected programming language is **C++14**. We emphasize the submission of a fully working code over partially correct but efficient code. Once **submitted**, you cannot review this problem again. You can use `cout` to debug your code. The `cout` may not work in case of syntax/runtime error. The version of **GCC** being used is **5.5.0**.

A pizza shop makes serves both vegan pizza and meat pizza. Customers place N orders at the shop. Their order number is printed on their bill. The shop displays K out of N orders (both vegan and meat) on their display screen at any given time. The pizza shop is very famous and they receive many orders. To avoid confusion, vegan pizza orders are represented by positive numbers and meat pizza orders are represented by negative numbers on the screen. The orders are served in the order in which they are displayed on the screen. Each time an order is ready, its number is

```
3 #include<string>
4 #include<vector>
5 using namespace std;
6
7
8 /*
9 *
10 */
11 vector<int> orderPizza (vector<int> orderPlaced, int size)
12 {
13     vector<int> answer;
14     // Write your code here
15
16     return answer;
17 }
18
19
20 int main()
21 {
22
23     //input for orderPlaced
24     int orderPlaced_size;
25     cin >> orderPlaced_size;
26     vector<int> orderPlaced;
27     for ( int idx = 0; idx < orderPlaced_size; idx++ )
28     {
29         int temp;
30         cin >> temp;
31         orderPlaced.push_back(temp);
32     }
33     //input for size
34     int size;
35     cin >> size;
```

Test Case

Question

are displayed on the screen. Each time an order is ready, its number is removed from the display screen and a new order is added to the display at the end of the list.

A couple has arrived with their child Billy. Billy is a very naughty child. To keep him occupied, his parents tell him to make a list of the first meat pizza that appear in each set of K orders displayed on the shop's display screen.

Write an algorithm to help Billy make a list of the first meat based pizza order numbers displayed on the screen each time an order is delivered to a customer.

Input

The first line of the input consists of an integer -

orderPlaced_size, representing the total number of orders placed (N). The second line consists of N space-separated integers representing the vegan pizza and meat pizza order numbers of the orders placed by

```
4 #include<vector>
5 using namespace std;
6
7
8 /*
9 *
10 */
11 vector<int> orderPizza (vector<int> orderPlaced, int size)
12 {
13     vector<int> answer;
14     // Write your code here
15
16
17     return answer;
18 }
19
20 int main()
21 {
22
23     //input for orderPlaced
24     int orderPlaced_size;
25     cin >> orderPlaced_size;
26     vector<int> orderPlaced;
27     for ( int idx = 0; idx < orderPlaced_size; idx++ )
28     {
29         int temp;
30         cin >> temp;
31         orderPlaced.push_back(temp);
32     }
33     //input for size
34     int size;
35     cin >> size;
```

Test

Question

an integer -

orderPlaced_size, representing the total number of orders placed (N). The second line consists of N space-separated integers representing the vegan pizza and meat pizza order numbers of the orders placed by the customers.

The last line consists of an integer - *size*, representing the number of orders displayed on the screen (K).

Output

Print space-separated integers representing the first meat pizza order of every K orders displayed on the screen each time an order is delivered to a customer. Print 0 if the screen does not display any meat pizza order.

Constraints

$0 \leq \text{orderPlaced_size} \leq 10^6$
 $0 \leq \text{size} \leq \text{orderPlaced_size}$
 $-10^9 \leq$ order numbers of the orders placed $\leq 10^9$

Example

Code saved

```
4 #include<vector>
5 using namespace std;
6
7
8 /*
9 *
10 */
11 vector<int> orderPizza (vector<int> orderPlaced, int size)
12 {
13     vector<int> answer;
14     // Write your code here
15
16
17     return answer;
18 }
19
20 int main()
21 {
22
23     //input for orderPlaced
24     int orderPlaced_size;
25     cin >> orderPlaced_size;
26     vector<int> orderPlaced;
27     for ( int idx = 0; idx < orderPlaced_size; idx++ )
28     {
29         int temp;
30         cin >> temp;
31         orderPlaced.push_back(temp);
32     }
33     //input for size
34     int size;
35     cin >> size;
```

Test C

Question

meat pizza order.

Constraints

 $0 \leq \text{orderPlaced_size} \leq 10^6$ $0 \leq \text{size} \leq \text{orderPlaced_size}$ $-10^9 \leq \text{order numbers of the orders}$ $\text{placed} \leq 10^9$

Example

Input:

6
-11 -2 19 37 64 -18
3

Output:

-11 -2 0 -18

Explanation:

Step 1: At the time of the first display, the order numbers displayed are [-11, -2, 19].

So, the first displayed meat pizza order number is -11.

Step 2: At the time of the next display, the order numbers displayed are [-2, 19, 37].

So, the first displayed meat pizza order number is -2.

```
4 #include<vector>
5 using namespace std;
6
7
8 /*
9 *
10 */
11 vector<int> orderPizza (vector<int> orderPlaced, int size)
12 {
13     vector<int> answer;
14     // Write your code here
15
16
17     return answer;
18 }
19
20 int main()
21 {
22
23     //input for orderPlaced
24     int orderPlaced_size;
25     cin >> orderPlaced_size;
26     vector<int> orderPlaced;
27     for ( int idx = 0; idx < orderPlaced_size; idx++ )
28     {
29         int temp;
30         cin >> temp;
31         orderPlaced.push_back(temp);
32     }
33     //input for size
34     int size;
35     cin >> size;
```

Test C

Question

3

Output:

-11 -2 0 -18

Explanation:

Step 1: At the time of the first display, the order numbers displayed are [-11, -2, 19].

So, the first displayed meat pizza order number is -11.

Step 2: At the time of the next display, the order numbers displayed are [-2, 19, 37].

So, the first displayed meat pizza order number is -2.

Step 3: At the time of the next display, the order numbers displayed are [19, 37, 64].

Since no meat based pizza order is displayed, the output is 0.

Step 4: Similarly, at the time of the next display, the order numbers displayed are [37, 64, -18].

So, the first displayed meat pizza order number is -18.
So, the output is [-11, -2, 0, -18].

```
4 #include<vector>
5 using namespace std;
6
7
8 /*
9  *
10 */
11 vector<int> orderPizza (vector<int> orderPlaced, int size)
12 {
13     vector<int> answer;
14     // Write your code here
15
16
17     return answer;
18 }
19
20 int main()
21 {
22
23     //input for orderPlaced
24     int orderPlaced_size;
25     cin >> orderPlaced_size;
26     vector<int> orderPlaced;
27     for ( int idx = 0; idx < orderPlaced_size; idx++ )
28     {
29         int temp;
30         cin >> temp;
31         orderPlaced.push_back(temp);
32     }
33     //input for size
34     int size;
35     cin >> size;
```

Test C

```
4 #include<vector>
5 using namespace std;
6
7
8 /*
9 *
10 */
11 vector<int> orderPizza (vector<int> orderPlaced, int size)
12 {
13     vector<int> answer;
14     // Write your code here
15
16     return answer;
17 }
18
19
20 int main()
21 {
22
23     //input for orderPlaced
24     int orderPlaced_size;
25     cin >> orderPlaced_size;
26     vector<int> orderPlaced;
27     for ( int idx = 0; idx < orderPlaced_size; idx++ )
28     {
29         int temp;
30         cin >> temp;
31         orderPlaced.push_back(temp);
32     }
33     //input for size
34     int size;
35     cin >> size;
```

Test Cases & Output

▶ RUN SOLUTION



Search



ASUS Vivobook


```
17     return answer;
18 }
19
20 int main()
21 {
22     //input for orderPlaced
23     int orderPlaced_size;
24     cin >> orderPlaced_size;
25     vector<int> orderPlaced;
26     for ( int idx = 0; idx < orderPlaced_size; idx++ )
27     {
28         int temp;
29         cin >> temp;
30         orderPlaced.push_back(temp);
31     }
32     //input for size
33     int size;
34     cin >> size;
35
36
37     //output
38     vector<int> result = orderPizza(orderPlaced, size);
39     for ( int idx = 0; idx < result.size() - 1; idx++ )
40     {
41         cout << result[idx] << " ";
42     }
43     cout << result[result.size() - 1];
44
45     return 0;
46 }
47
48
```

Test Cases & Output

▶ RUN SOLUTION



Search



ASUS Vivobook

SHL

Question

The current selected programming language is **C**. We emphasize the submission of a fully working code over partially correct but efficient code. Once **submitted**, you cannot review this problem again. You can use `printf()` to debug your code. The `printf()` may not work in case of syntax/runtime error. The version of **GCC** being used is **5.5.0**.

A student must solve an entire workbook of problems related to finding the area of intersection of two circles. Because the problems are all very similar, the student decides to write a program that can solve all these similar problems.

Input

The first line of the input consists of an integer $centerX_1$, representing the x coordinates of the center of the first circle.
The second line consists of an integer $centerY_1$, representing the y coordinates of the center of the first circle.
The third line consists of an integer $radius_1$, representing the radius of the first circle.
The fourth line consists of an integer $centerX_2$, representing the x coordinates of the center of the second circle.
The fifth line consists of an integer $centerY_2$, representing the y coordinates of the center of the second circle.
The last line consists of an integer $radius_2$, representing the radius of the second circle.

Output

Print a real number representing the area of intersection of two circles rounded up to 6 decimal places.

Constraints

$0 < radius_1, radius_2 < 10^4$

Example

Input:

0
0
2
3
0
2

Output:

1.813247

Explanation:

First circle has center coordinates as (0,0) and radius 2.
Second circle has center coordinates as (3,0) and radius 2.
The area of intersection of two circles is 1.813247.

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41


```
1 // Header Files
2 #include<iostream>
3 #include<string>
4 #include<vector>
5 using namespace std;
6
7
8 /*
9  *
10 */
11 double areaOfIntersection (int centerX1, int centerY1, int radius1, int centerX2, int centerY2, int radius2)
12 {
13     double answer;
14     // Write your code here
15
16
17     return answer;
18 }
19
20 int main()
21 {
22     //input for centerX1
23     int centerX1;
24     cin >> centerX1;
25
26     //input for centerY1
27     int centerY1;
28     cin >> centerY1;
29
30     //input for radius1
31     int radius1;
32     cin >> radius1;
33
34     //input for centerX2
35     int centerX2;
36     cin >> centerX2;
37
38     //input for centerY2
39     int centerY2;
40     cin >> centerY2;
41
```


SHL

Question

The current selected programming language is **C++**. We emphasize the submission of a fully working code over partially correct but efficient code. Once **submitted**, you cannot review this problem again. You can use `cout` to debug your code. The `cout` may not work in case of syntax/runtime error. The version of **G++** being used is **5.5.0**.

A space organization has N satellites in orbit. The satellites are assigned IDs from 0 to $N-1$. A satellite can transfer data to any other satellite that lies within its bandwidth range.

A team at organization is working on a project involving satellite communications. In this project, they wish to transfer data from the main satellite with ID 0 to all the other satellites. But because the transfer of data requires an enormous amount of power, a satellite can transfer data to only one other satellite at a time. Then, the satellite can connect to the server at organization which will help to connect this satellite to other satellites near it. Only a fixed number of satellites can be in the bandwidth range of a satellite at a time. The team wishes to determine the minimum number of iterations of data transfer necessary to connect all the satellites.

Write an algorithm to help the team to determine the minimum number of iterations of data transfer necessary to connect all the satellites.

Input

The first line of the input consists of two space-separated integers $numSatellite$ (N) and $constX$, representing the number of satellites and $constX$ is always 2.

The next $N-1$ lines consist of $constX$ space-separated integers representing the satellite that can transfer data and the satellite that can receive the data, respectively.

The last line consists of an integer $maxSatellites$ representing the maximum number of satellites that can be connected to one satellite.

Output

Return an integer representing the minimum number of iterations of data transfer necessary to connect all the satellites.

Constraints

$$0 < numSatellite \leq 10^4$$

$$0 < maxSatellites \leq numSatellite$$

Example

Input:

```
9 2
0 1
0 2
0 3
1 4
2 5
2 6
2 7
3 8
3 9
```



Question

satellite can transfer data to only one other satellite at a time. Then, the satellite can connect to the server at organization which will help to connect this satellite to other satellites near it. Only a fixed number of satellites can be in the bandwidth range of a satellite at a time. The team wishes to determine the minimum number of iterations of data transfer necessary to connect all the satellites.

Write an algorithm to help the team to determine the minimum number of iterations of data transfer necessary to connect all the satellites.

Input

The first line of the input consists of two space-separated integers *numSatellite* (N) and *constX*, representing the number of satellites and *constX* is always 2.

The next N-1 lines consist of *constX* space-separated integers representing the satellite that can transfer data and the satellite that can receive the data, respectively.

The last line consists of an integer *maxSatellites* representing the maximum number of satellites that can be connected to one satellite.

Output

Return an integer representing the minimum number of iterations of data transfer necessary to connect all the satellites.

Constraints

$$0 < \text{numSatellite} \leq 10^4$$

$$0 < \text{maxSatellites} < \text{numSatellite}$$

Example

Input:

```
9 2
0 1
0 2
0 3
1 4
2 5
2 6
2 7
3 8
3
```

Output:

```
4
```

Explanation:

The main satellite with ID 0 will transfer the data to one of the satellites connected to it i.e. 1.
In the second iteration, the satellites 0 and 2 transfer data to satellites 3 and 4 respectively.
In the third iteration, the satellites 0, 2 and 1 transfer data to satellites 5, 6 and 7 respectively.
In the fourth iteration, the satellites 2 and 3 transfer data to satellites 8 and 9 respectively.
So, all the satellites get connected after four iterations.

14

25

26

27

38

3

Output:

4

Explanation:

The main satellite with ID 0 will transfer the data to one of the satellites connected to it (i.e., 2).

In the second iteration, the satellites 0 and 2 transfer data to satellites 1 and 5, respectively.

In the third iteration, the satellites 0, 2 and 1 transfer data to satellites 3, 6 and 4, respectively.

In the fourth iteration, the satellites 2 and 3 transfer data to satellites 7 and 8, respectively.

So, all the satellites get connected after four iterations.




```

1 // Header Files
2 #include
3 #include
4 #include<vector>
5 using namespace std;
6
7
8 /*
9  * connections, represents connection between the satellites.
10 maxSatellites, represents the maximum number of satellites that can be connected to one satellite.
11 */
12 int maxIterations (vector<vector<int> > connections, int maxSatellites)
13 {
14     int answer;
15     // Write your code here
16
17
18     return answer;
19 }
20
21 int main()
22 {
23     //input for connections
24     int connections_row;
25     int connections_col;
26     cin >> connections_row;
27     cin >> connections_col;
28
29     vector<vector<int> > connections;
30     for ( int idx = 0; idx < connections_row-1; idx++ )
31     {
32         vector<int> temp_vector;
33         for ( int jdx = 0; jdx < connections_col; jdx++ )
34         {
35             int temp;
36             cin >> temp;

```